



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

PROYECTO DE CARRERA: INGENIERÍA INFORMÁTICA

UNIDAD CURRICULAR : LENGUAJES Y COMPILADORES

SEMESTRE VII – 2026 – I

Los Lenguajes de Programación

Profesor:

Marquez, Felix

Estudiantes:

Frontado, Keiner C.I. 32.059.780

Martínez, Giovanni C.I. 31.782.337

Morales, Edgar C.I. 30.631.492.

Ortega, Mansour C.I. 28.785.297

Ciudad Guayana, junio 2026

Introducción

En el ejercicio de la Ingeniería en Informática, la interacción con los lenguajes de programación suele abordarse inicialmente desde la perspectiva del software aplicativo, donde cada herramienta es seleccionada por su enfoque, sintaxis, paradigma o por su especialización en un área, como la orientación a objetos o la gestión de bases de datos. Sin embargo, para alcanzar una comprensión profunda de la ciencia detrás de estos lenguajes, es indispensable desplazar la mirada del simple uso del lenguaje hacia la infraestructura interna que lo sustenta. Es crucial comprender cómo estos sistemas procesan los componentes léxicos (tokens) y traducen instrucciones complejas en un código que el computador pueda interpretar y ejecutar de manera eficiente.

El presente trabajo de investigación tiene como objetivo principal estudiar la arquitectura y los fundamentos teóricos detrás de los lenguajes de programación, evaluando cómo las decisiones de diseño léxico, morfológico y sintáctico impactan directamente en su rendimiento y en sus mecanismos de ejecución. Para cumplir con este propósito, el informe aborda tres fases esenciales: en primer lugar, el análisis analítico de la convergencia multiparadigma en el software moderno; en segundo lugar, una medición empírica de rendimiento (benchmarking) que contrasta el comportamiento de diversos entornos de desarrollo ante algoritmos de procesamiento intensivo; y finalmente, la especificación formal de un Lenguaje de Dominio Específico (DSL) diseñado para la automatización y control seguro de microrredes eléctricas inteligentes y almacenamiento de energía (ECO-GRID).

A través de este recorrido, el lector e investigador podrán asimilar de forma práctica y rigurosa los elementos estructurales que dan vida a un lenguaje de programación, tales como el alfabeto aceptado, las reglas morfológicas, la clasificación exhaustiva de tokens, las palabras clave operacionales y la construcción de gramáticas formales.

Conceptos Fundamentales

Para no irnos de lleno con el desglose experimental e investigativo de las actividades informáticas, es crucial definir conceptualmente ciertos términos de la teoría de lenguajes y autómatas que forman parte de los cimientos de nuestra disciplina:

Abstracción Lingüística: En la ciencia de la computación, representa el nivel de distanciamiento que existe entre la sintaxis de un lenguaje de programación y las instrucciones de código máquina binario que ejecuta directamente el procesador. Los lenguajes de alto nivel elevan la abstracción para favorecer la productividad humana gracias a la cercanía del lenguaje con el que nos comunicamos, delegando en el compilador la tarea de traducir dicha semántica abstracta hacia operaciones físicas de registros y memoria.

Traducción Computacional (Compilación vs. Interpretación): Es el proceso de transformación de un programa escrito en un lenguaje de alto nivel (código fuente) a un formato ejecutable por la máquina. Técnicamente, se dividen en dos filosofías: la *compilación*, que traduce de forma íntegra el código antes de su ejecución generando un archivo nativo binario; y la *interpretación*, que analiza y ejecuta las instrucciones línea por línea en tiempo de ejecución a través de una máquina virtual o entorno intermedio.

Morfología Léxica: Es el área de la teoría de lenguajes de programación que se encarga de estudiar las unidades mínimas con significado propio dentro de un código fuente, conocidas formalmente como *tokens*. El análisis léxico ignora las jerarquías estructurales y las reglas de orden, concentrándose únicamente en validar que los caracteres ingresados (letras, números, símbolos) pertenecen al alfabeto aceptado y se agrupan bajo los patrones morfológicos permitidos por el diseño del lenguaje.

Sintaxis Estructurada: A diferencia de la morfología, la sintaxis define las reglas gramaticales abstractas que gobiernan el orden jerárquico y la combinación de los tokens para que una instrucción o sentencia sea considerada válida por el compilador. La sintaxis asegura que el flujo conceptual del código posea un orden lógico incuestionable, de modo que la máquina pueda interpretar el árbol de comandos sin dar pie a ambigüedades operacionales.

Una vez delimitados los cimientos conceptuales que rigen la abstracción y la morfología de las herramientas de software, el cuerpo central de este informe se despliega en tres fases consecutivas y complementarias. En la sección de **Desarrollo**, la **Actividad I** iniciará con un examen exhaustivo de la programación paradigmática de programación, evaluando cómo las corrientes históricas y modernas modelan el flujo de datos. Posteriormente, la **Actividad II** someterá estas abstracciones a una rigurosa prueba empírica de rendimiento mediante un proceso de *benchmarking*. Finalmente, la **Actividad III** consolidará el conocimiento adquirido a través de la síntesis formal del *Lenguaje L*, llevando la teoría de los compiladores al terreno práctico de la automatización industrial.

Actividad I: Matriz Descriptiva y Análisis de Paradigmática

El concepto de paradigma, suele ser visto como sinónimo de “ejemplo” o “manual” de algo, sin embargo, el día de hoy vamos a utilizar la definición formal que se le dá a este término en la disciplina científica, la cual hace referencia a el conjunto de teorías, leyes y técnicas que una comunidad científica acepta en una época determinada, en este sentido, en la presente investigación se tomará un paradigma como el modelo o patrón que se usa para entender o resolver problemas en la disciplina de la programación.

Antes de abordar los paradigmas del panorama actual de la programación, debemos tener en cuenta que además de los paradigmas que veremos a continuación existe el “paradigma personal de programación”, el cual, más allá de los paradigmas fundamentales de la programación, está relacionado con el propio desarrollador, siendo la manera o filosofía del programador a la hora de desarrollar código o resolver problemas, sin limitarse a un solo lenguaje, este paradigma puede ser una combinación de técnicas según prioridades, la forma de cada quien para afrontar problemas y nuestra manera de estructurar proyectos; así brindando una estandarización de nuestro trabajo y facilitando el mantenimiento

En la actualidad, disponemos de muchas herramientas diseñadas para agilizar nuestro trabajo, en este sentido, con el uso adecuado de los paradigmas de la programación logramos el máximo desempeño a la hora de usar estas herramientas, no se deben ver como reglas limitantes, sino como manuales o enfoques arquitectónicos para resolver problemas de manera eficiente.

Podemos separar los paradigmas de manera cronológica en 2 bloques principales: los paradigmas clásicos y los paradigmas emergentes o consolidados en la modernidad. De esta forma debemos entender que los paradigmas clásicos, son a su vez los fundamentos que debemos aprender cuando ingresamos al mundo de la programación.

Entre los paradigmas clásicos podemos encontrar:

- **Paradigma Imperativo/Estructural:** Este es el enfoque más cercano a la arquitectura von Neumann de las computadoras, por estar basado en dar instrucciones directas, explícitas y secuenciales a la máquina, en las cuales se le detalla el cómo debe cambiar su estado. En este sentido el programa se define como una secuencia de

instrucciones paso a paso, por lo cual, la modularidad se logra mediante funciones o procedimientos con los cuales se encapsulan bloques de código; su mecanismo de estado resalta por su alta mutabilidad, pues las variables cambian de valor constantemente. En la actualidad se usa para desarrollo de sistemas operativos, programación de microcontroladores y sistemas embebidos, y en el desarrollo de videojuegos de bajo nivel para mejorar el rendimiento y optimizar recursos; siendo esta una de sus ventajas principales, la optimización de memoria, la alta velocidad de ejecución y el control absoluto del software. Por otro lado tiene ciertas limitaciones a la hora de escalar el software, rastrear variables mutadas globalmente produce fenómenos como el conocido “código espagueti”, un caso de estudio popular de esto es el videojuego “League of Legends” software de “Riot Games, Inc.”, escalar durante años el software de un proyecto “indie” ha ocasionado que en su código existan dependencias inútiles que afectan el funcionamiento y se gasten recursos tratando de solucionarlo.

- **Paradigma Orientado a objetos:** Nacido a partir de la necesidad de solventar la crisis del software de los años 70 y 80, este enfoque permite modelar el software a partir de conceptos del mundo real mediante “objetos”. Está basado en 4 pilares fundamentales, como lo son: la abstracción, que consiste en aislar las características esenciales de un elemento; el encapsulamiento, que se trata de ocultar el estado interno y obligar a interactuar solo a través de interfaces públicas conocidas como métodos; la herencia, con la cual podemos reutilizar y extender código de clases padre a clases hijas; y el polimorfismo, que consiste en la capacidad de que diferentes objetos puedan responder al mismo mensaje de maneras distintas. Debido a su orientación a objetos, su mecanismo de estado es mutable pero controlado, debido a que el estado reside dentro del objeto y solo sus propios métodos deben poder modificarlo; en la actualidad se usa en grandes sistemas empresariales basados principalmente en Java o C#, en el desarrollo de backend robusto, arquitecturas monolíticas y frameworks de desarrollo de videojuegos. Entre sus ventajas se encuentra la reutilización de código, la organización para equipos grandes mediante patrones de diseño y la facilidad del mantenimiento en dominios de negocios complejos; por otro lado también puede conducir a jerarquías de herencia excesivamente complejas, dificultades para manejar concurrencia nativa debido al estado mutable y sobrecarga de memoria; un caso de estudio al respecto, es el motor

gráfico Unreal Engine hecho en C++, si bien facilita el desarrollo de los videojuegos, no está bien optimizado debido a su propia arquitectura y su diseño.

En la actualidad, siguen surgiendo paradigmas a la vez que otros se consolidan, entre estos paradigmas modernos podemos encontrar:

- **Paradigma funcional:** Se inspira en el cálculo lambda matemático y trata a la computación como la evaluación de matemáticas puras y evita los datos mutables, en este sentido está apoyada en tres conceptos críticos: las funciones puras, que consisten en que el mismo input siempre produce el mismo output; la inmutabilidad, implica que los datos no se modifican, solo se transforman para crear nuevos datos; las funciones de primera clase, las funciones se pueden pasar como argumentos, retornar de otras funciones o almacenar en variables. Su mecanismo de estado es estrictamente inmutable como se mencionó anteriormente, actualmente se usa en Pipelines de Big Data y análisis de datos, procesamiento de transacciones financieras concurrentes y manipulación de estados en el desarrollo web frontend. Entre sus ventajas radica que es muy fácil realizar pruebas unitarias, su código es predecible y se pueden eliminar nativamente condiciones de carrera en entornos concurrentes o distribuidos; sin embargo conlleva una curva de aprendizaje empinada para desarrolladores previamente acostumbrados al pensamiento imperativo, también este puede generar una alta recolección de datos basura si no se utiliza el manejo de la memoria.
- **Paradigma declarativo/lógico:** A diferencia del enfoque imperativo, el declarativo se basa en describir qué resultado se desea obtener, analizando de forma aislada el proceso de cómo la computadora lo calcula, el desarrollador define las restricciones, reglas o estructura del software con lo cual un motor interno o intérprete se encarga de ejecutar los pasos imperativos subyacentes. su mecanismo de estado es abstraído, por lo cual el desarrollador no gestiona el flujo de estado directamente; actualmente se usa en consultas de bases de datos relacionales (como SQL), maquetación y estilos web (como HTML/CSS), herramientas de infraestructura como código y desarrollo moderno de interfaces de usuario interactivas (como React y Flutter). Entre sus ventajas encontramos su extrema expresividad, la reducción masiva de líneas de código repetitivo y la facilidad en la lectura; no obstante se pierde el control fino

sobre el rendimiento, pues si el motor que interpreta las instrucciones es ineficiente, optimizar la ejecución desde el código del usuario es complicado.

- **Paradigma concurrente:** Este paradigma surge como respuesta a la necesidad física y arquitectónica de la computación moderna, hablamos de los límites de velocidad de los procesadores mononúcleo y el fin de la ley de Moore, y la necesidad de sistemas altamente disponibles que no bloqueen la experiencia de usuario. La computación concurrente descompone un programa en unidades de ejecución independientes que pueden ejecutarse de forma intercalada en un solo núcleo compartiendo tiempo de CPU o en paralelo de forma simultánea en varios núcleos para maximizar el uso del procesador y gestionar tareas que dependen de tiempos de espera extensos; se maneja un mecanismo de estado crítico y altamente complejo, en la concurrencia, supone un desafío el acceso simultáneo a la memoria compartida, si dos hilos intentan modificar la misma variable a la vez, ocurre una condición de carrera y se corrompen los datos. En la actualidad es usado como motor de bases de datos relacionales y no relacionales por igual, gestionando miles de consultas por segundo, usualmente se ve en servidores web de alto rendimiento y en sistemas operativos administrando los recursos del hardware y los procesos del usuario en segundo plano. Todo indica a que su mayor ventaja es la alta eficiencia en el uso de hardware multinúcleo, así reduciendo de forma drástica los tiempos de respuesta y previniendo bloqueos en las interfaces de usuario; sin embargo su diseño es sumamente complejo, haciendo que depurar errores temporales o intermitentes sea sumamente difícil, debido a que el orden de ejecución exacto depende del planificador (scheduler) del sistema operativo y no del programador.

Paradigma	Gestión del Estado y Secuenciación	Mutabilidad y Memoria	Abstracción y Características Clave	Mitigación de Efectos Secundarios / Control
Imperativo / Estructural	Explícita e instruccional. El flujo es lineal y determinado por cambios secuenciales de estado mediante estructuras de control (for, while, if).	Alta mutabilidad. Las variables representan celdas de memoria física modificables directamente en cualquier momento.	Procedimental. Centrado en algoritmos paso a paso que operan sobre datos separados de la lógica.	Ninguna. Los efectos secundarios son nativos y esperados; requiere disciplina del programador para evitar colisiones de datos.
Orientado a Objetos (POO)	Dirigida por mensajes. El estado cambia cuando los objetos invocan métodos entre sí, alterando su comportamiento interno.	Mutabilidad controlada. El estado está encapsulado; la memoria sólo se altera a través de las interfaces o métodos públicos autorizados.	Modelado del dominio. Basado en encapsulamiento, polimorfismo, herencia y composición.	Aislamiento por visibilidad. El uso de modificadores (private, protected) restringe qué componentes pueden generar efectos secundarios.
Funcional	Evaluación de expresiones. El flujo se rige por la aplicación de funciones matemáticas puras; soporta evaluación perezosa (lazy evaluation).	Inmutabilidad estricta. No existen las variables tradicionales; cualquier "cambio" genera una nueva estructura de datos en memoria.	Funciones de primer orden. Las funciones se tratan como datos ordinarios; alta confianza en la transparencia referencial.	Eliminación programática. Se prohíbe la alteración del entorno externo, erradicando por diseño los efectos colaterales.

Lógico / Declarativo	Inferencia automática. Basado en relaciones lógicas. El flujo de control está completamente abstraído por el motor de ejecución.	Gestión implícita. No hay noción de asignación de memoria destructiva; se maneja mediante ligadura de variables únicas.	Cláusulas de Horn. Se compone de hechos, reglas lógicas y el proceso algorítmico de unificación para resolver consultas.	Abstracción total. Al no haber instrucciones de bajo nivel ni mutación, el concepto de efecto secundario no existe en el código.
Concurrente / Actores	Asíncrona por el paso de mensajes. Los actores procesan de forma aislada las peticiones acumuladas en sus colas virtuales.	Confinamiento estricto. Cada actor posee su propio espacio de memoria. Está prohibido compartir datos mutables entre hilos	Aislamiento de entidades. Arquitectura distribuida donde el sistema se compone de múltiples micro-unidades autónomas.	Mitigación lingüística. Al eliminar la memoria compartida, se neutralizan las condiciones de carrera a nivel de diseño del lenguaje.

Actividad II: Estudio Morfológico, sintáctico y benchmarking

Para profundizar en la vertiente empírica de esta investigación, la Actividad II se centra en el estudio comparativo de cuatro lenguajes de programación distintos: JavaScript, Python, Rust y Zig. Para este propósito, se ha desarrollado un algoritmo de cálculo basado en la conjetura de Collatz, el cual actúa como escenario de prueba para evaluar la eficiencia de cada entorno. A continuación, se presenta un desglose analítico de las características morfológicas y sintácticas inherentes a cada lenguaje, complementado por un análisis exhaustivo de benchmarking que permite contrastar el rendimiento computacional y la gestión de recursos de estas herramientas ante un mismo algoritmo de procesamiento iterativo.

Morfología y Semántica en JavaScript

JavaScript, es un lenguaje interpretado y de tipado dinámico que se rige bajo ECMAScript, la especificación estándar que define las normas del lenguaje, posee una gramática muy flexible pensada originalmente para la interoperabilidad en entornos web y dominios de objetos dinámicos. El estándar define un vocabulario estricto de palabras reservadas que encapsulan abstracciones de control de flujo, administración de memoria declarativa y paradigmas de orientación a objetos.

El escáner léxico bloquea la utilización de identificadores que correspondan a tokens como `abstract`, `arguments`, `boolean`, `break`, `byte`, `case`, `catch`, `char`, `class`, `const`, `continue`, `debugger`, `default`, `delete`, `do`, `double`, `else`, `enum`, `eval`, `export`, `extends`, `false`, `final`, `finally`, `float`, `for`, `function`, `goto`, `if`, `implements`, `import`, `in`, `instanceof`, `int`, `interface`, `let`, `long`, `native`, `new`, `null`, `package`, `private`, `protected`, `public`, `return`, `short`, `static`, `super`, `switch`, `synchronized`, `this`, `throw`, `throws`, `transient`, `true`, `try`, `typeof`, `var`, `void`, `volatile`, `while`, `with`, `yield`. La presencia de estas palabras reserva arquitecturas lógicas específicas en la fase de análisis sintáctico, permitiendo que el motor de ejecución delimite los contextos léxicos y el alcance de las variables.

Mediante el uso de expresiones regulares, el escáner de JavaScript logra identificar y agrupar diversos caracteres Unicode tales como el avance de línea (`\n`), el retorno de carro (`\r`) y los separadores de líneas o párrafos, junto a las barras espaciadoras y tabulaciones bajo la clasificación técnica de espacios en blanco. En una etapa previa a la interpretación, el sistema también rastrea y elimina del flujo de ejecución tanto los comentarios de una sola línea (`//`)

como los de bloque (`/* */`). Un aspecto distintivo de la morfología de ECMAScript es la Inserción Automática de Punto y Coma (ASI); este mecanismo permite que el escáner analice la geometría sintáctica y los terminadores de línea para inyectar tokens de punto y coma de forma silenciosa. Dicha funcionalidad garantiza la corrección morfológica del código incluso ante la falta de terminadores explícitos, aunque introduce el riesgo de generar ambigüedades estructurales que otros lenguajes más estrictos evitarían mediante errores de análisis.

El analizador morfológico de JavaScript carece de términos para designar enteros de bits variables, gestionando literales como punto flotante de 64 bits (IEEE 754). Para optimizar esto, V8 Engine emplea la técnica Smi (*Small Integer*), que etiqueta virtualmente valores de 31 bits con signo para que residan directamente en punteros de memoria, evitando asignaciones en el montón. No obstante, si la variable n supera este límite en la conjetura de Collatz, V8 promociona el valor a *HeapNumber*, saturando la memoria con asignaciones de 64 bytes y degradando el rendimiento.

Morfología y Semántica en Python

Python es un lenguaje interpretado y de tipado dinámico que se rige bajo la Referencia del Lenguaje Python, la especificación técnica que define las normas del lenguaje, y posee una gramática diseñada para priorizar la legibilidad y la estructura espacial del código. El estándar define un vocabulario estricto de palabras reservadas que encapsulan abstracciones de control de flujo y administración de lógica.

La estructura física del texto fuente está separada fundamentalmente de la estructura lógica de ejecución. Una línea lógica de código Python culmina de manera restrictiva con la emisión interna del token NEWLINE. El espacio en blanco se eleva a la categoría de elemento léxico imperativo en Python. El motor de tokenización vigila meticulosamente la cantidad de espacios métricos al inicio de una línea de caracteres. Cuando esta métrica incrementa en relación a la declaración precedente, el escáner inyecta el token de control INDENT, determinando la apertura de un nuevo bloque lógico o bucle de ámbito. Por contrapartida, cuando la sangría se retrae, el sistema genera secuencialmente uno o múltiples tokens DEDENT hasta nivelar el árbol.

El tratamiento del espacio libre intermedio entre tokens en una misma línea horizontal, sin embargo, es ignorado de forma habitual. Los comentarios, prefijados unívocamente por el carácter de almohadilla o numeral (`#`), dictaminan el final inminente de la línea física para el analizador sintáctico y son completamente descartados del árbol interno,

a menos que el flujo de procesamiento dispare reglas de unión de líneas implícitas, operables al anidar código dentro de llaves, corchetes o cadenas de documentación de comillas triples.

El escáner léxico bloquea la utilización de identificadores que correspondan a tokens como `False`, `class`, `finally`, `is`, `return`, `None`, `continue`, `for`, `lambda`, `try`, `True`, `def`, `from`, `nonlocal`, `while`, `and`, `del`, `global`, `not`, `with`, `as`, `elif`, `if`, `or`, `yield`, `assert`, `else`, `import`, `pass`, `break`, `except`, `in`, y `raise`. A nivel léxico, Python también define clases particulares de identificadores reservados que se manifiestan mediante la inclusión de guiones bajos iniciales o finales (por ejemplo, identificadores mágicos), los cuales alteran la resolución de importaciones y la herencia de objetos subyacentes en el intérprete.

En CPython, todo componente léxico que represente un literal numérico induce la invocación de una macro de asignación para instanciar una estructura en memoria C sumamente compleja denominada `PyLongObject`. Esta arquitectura abstraída no guarda su valor numérico en un único registro escalar binario, sino que se define como un arreglo de dígitos matemáticos implementados en base 2^{15} o base 2^{30} , dependiendo estrictamente de la capa de compilación de la plataforma C inferior. El `PyLongObject` se adjunta a un pesado encabezado de control que almacena el tamaño del propio arreglo y rastrea minuciosamente el contador global de referencias asociadas al objeto para coordinar al recolector de basura nativo.

Morfología y Semántica en Rust

Rust se define como un lenguaje de programación compilado de tipado estático, regido formalmente por la Referencia del Lenguaje Rust, la cual establece las normativas técnicas orientadas a la garantía de seguridad en memoria y la gestión eficiente de la concurrencia. Su arquitectura gramatical ha sido concebida bajo principios de diseño pragmático que priorizan la verificabilidad absoluta y la reducción de ambigüedades semánticas durante el proceso de compilación Ahead-Of-Time. El motor de análisis léxico descompone el flujo de código fuente en unidades discretas de tokens, permitiendo el procesamiento riguroso de directivas operacionales, símbolos de puntuación y estructuras de anidación complejas.

A diferencia de otros entornos, el analizador de Rust emplea el espacio en blanco únicamente como delimitador secuencial para identificadores, despojándolo de cualquier relevancia métrica en la definición de bloques lógicos. Tanto las anotaciones funcionales (`//`) como la documentación formal de bloque (`/* */`) son detectadas y disueltas en la etapa

precursora de la traducción, asegurando que el árbol sintáctico resultante permanezca libre de contaminación textual. Asimismo, el sistema impone de manera mandatoria la utilización del punto y coma como terminador explícito de instrucciones, reforzando la disciplina estructural del software.

El escáner léxico restringe el uso de identificadores que coincidan con palabras reservadas tales como `as`, `async`, `await`, `break`, `const`, `continue`, `crate`, `dyn`, `else`, `enum`, `extern`, `false`, `fn`, `for`, `if`, `impl`, `in`, `let`, `loop`, `macro`, `match`, `mod`, `move`, `mut`, `pub`, `ref`, `return`, `self`, `Self`, `static`, `struct`, `super`, `trait`, `true`, `type`, `unsafe`, `use`, `where`, `while`, y `yield`. Este análisis morfológico permite la integración de sufijos de precisión técnica directamente en los literales, facilitando la gestión de primitivas como `u8`, `u16`, `u32`, `u64`, `u128`, además de registros de longitud nativa (`usize`, `isize`) y especificaciones de punto flotante de alta fidelidad (`f32`, `f64`).

En la implementación de algoritmos iterativos, el empleo estricto de tipos `u64` otorga a la infraestructura de compilación (LLVM) los fundamentos necesarios para ejecutar optimizaciones de alto rendimiento. Al recibir instrucciones deterministas exentas de sobrecarga abstracta o mutación dinámica, el sistema genera código binario puro en ensamblador x86-64 de forma altamente sinérgica. Las operaciones de desplazamiento de bits para la aritmética de división se ejecutan de forma inmediata en la unidad lógica del procesador, materializando el concepto de abstracciones de costo cero fundamental en la industria del software moderno.

Morfología y Semántica en Zig

Zig es un lenguaje compilado de tipado estático que se rige bajo la Referencia del Lenguaje Zig, y cuya gramática está diseñada para garantizar la predictibilidad y la ausencia total de control de flujo oculto. La arquitectura del analizador léxico se ha diseñado bajo el enfoque de priorizar la mantenibilidad y la eliminación sistemática de cualquier gestión de memoria no explícita. Con el objetivo de suprimir ambigüedades semánticas en el proceso de compilación, el motor de escaneo descompone el código fuente mediante un esquema simplificado.

El analizador de Zig interpreta los espacios como delimitadores, pero impone una estructura donde la tokenización puede realizarse de forma mecánica línea por línea. En cuanto a las anotaciones, el diseño del analizador de Zig suprime los comentarios de bloque para simplificar el escaneo donde únicamente se admiten comentarios de línea denotados por

barras dobles (//) y comentarios documentales (///). Esta restricción garantiza que cualquier texto informativo se interrumpa en el salto de línea, eliminando problemas de análisis dependientes del contexto y asegurando una transformación predecible del texto fuente en tokens lógicos.

El escáner léxico bloquea la utilización de identificadores que correspondan a tokens como `align`, `allowzero`, `and`, `asm`, `async`, `await`, `break`, `catch`, `comptime`, `const`, `continue`, `defer`, `else`, `enum`, `errdefer`, `error`, `export`, `extern`, `fn`, `for`, `if`, `inline`, `noalias`, `noinline`, `nosuspend`, `opaque`, `or`, `orelse`, `packed`, `pub`, `resume`, `return`, `linksection`, `struct`, `suspend`, `switch`, `test`, `threadlocal`, `try`, `union`, `unreachable`, `usingnamespace`, `var`, `volatile`, `mientras` y `while`. La arquitectura de tipos permite designaciones de precisión milimétrica y tamaños arbitrarios que se comunican directamente con la ingeniería del procesador, facilitando una optimización agresiva del código binario resultante.

Zig destaca por su rechazo al asignador global implícito, exigiendo que cualquier gestión de memoria se realice mediante la inyección explícita de allocators. Esta transparencia elimina penalidades por recolección de basura o rastreo de referencias, permitiendo que el algoritmo opere exclusivamente sobre los registros y la pila estática. Al prescindir de un entorno de ejecución pesado, el código logra una eficiencia térmica y cronométrica máxima, procesando las transiciones de bits al límite de la capacidad del equipo sin desperdicio sistémico.

Análisis sintáctico por lenguaje

Debido a que Python presenta una gramática de alto nivel y una estructura espacial orientada a la legibilidad, este lenguaje se ha utilizado como referencia para la lógica del algoritmo de Collatz. Por consiguiente, el diseño del algoritmo inicial se realizó íntegramente en Python, sirviendo como modelo base para la adaptación lógica hacia los entornos de JavaScript, Rust y Zig. A continuación, se detalla la jerarquía sintáctica de las estructuras de control fundamentales empleadas en el código fuente, utilizando una notación formal basada en las reglas de producción.

Análisis sintáctico de Python

El código fuente en Python se caracteriza por el uso de la indentación como delimitador de bloques lógicos, sustituyendo los tokens de cierre explícitos por la profundidad del espacio en blanco. El algoritmo se organiza en tres niveles jerárquicos que dictan su ejecución:

Nivel de Control (simular_collatz): Actúa como el motor principal, invocando recursivamente al subprograma recursiva_collatz dentro de un bucle de control for que define el espacio muestral.

Nivel Decisorio (collatz): Opera como un nodo de bifurcación funcional. Su estructura condicional evalúa la paridad del valor mediante el operador módulo (%), retornando al estado siguiente del cálculo.

Nivel de Medición (benchmark_collatz): Encapsula la lógica de tiempo y memoria utilizando los módulos del sistema (tracemalloc, time), gestionando los contextos de ejecución de forma secuencial y acumulativa.

Análisis sintáctico de JavaScript

A diferencia de la rigidez espacial de Python, JavaScript (específicamente bajo el entorno de Node.js) emplea una gramática de estilo C, donde la delimitación de los bloques lógicos se gestiona mediante tokens de apertura ({) y cierre (}). El analizador sintáctico utiliza estos delimitadores para construir un Árbol de Sintaxis Abstracta que define explícitamente el alcance de cada estructura de control, independientemente de la sangría visual. La arquitectura del script en JavaScript se organiza en tres estratos funcionales que permiten la ejecución y medición del proceso:

Nivel de Control (simularCollatz): Constituye el nodo ejecutor central. A diferencia de la implementación en Python, este nivel gestiona explícitamente la memoria de salida mediante la instanciación de buffers (totalBuffer, lineaBuffer) para evitar cuellos de botella en la operación de escritura síncrona en el flujo estándar.

Nivel Decisorio (collatz): Actúa como el motor de cómputo puro. Utiliza operadores de igualdad estricta (===) y funciones estáticas de la clase Math para garantizar la integridad de los tipos de datos durante la transformación numérica, minimizando la conversión

implícita de tipos del motor V8.

Nivel de Medición (benchmarkCollatz): Encapsula el contexto de benchmarking utilizando la API `perf_hooks` y el gestor de memoria `process.memoryUsage()`. Este nivel incluye una invocación explícita al recolector de basura (`global.gc()`) antes de cada repetición para asegurar que las mediciones de RAM reflejen exclusivamente el consumo del algoritmo y no residuos de ejecuciones previas.

Análisis sintáctico de Rust

Rust se distingue por su arquitectura de tipado estático y su modelo de gestión de memoria basado en el sistema de propiedad (`ownership`), lo que impone una gramática altamente estricta y predecible. A diferencia de los entornos interpretados, el analizador sintáctico de Rust requiere una especificación explícita de tipos, límites de traits y estados de mutabilidad, elementos que son validados durante la etapa de compilación. Su sintaxis, aunque de herencia C, integra constructores modernos como las macros y los rangos de iteración, los cuales son procesados de forma eficiente por el compilador LLVM. La arquitectura del programa en Rust se organiza en tres estratos funcionales que garantizan la seguridad en memoria y el rendimiento de abstracción de costo cero:

Nivel de Control (simular_collatz): Actúa como el orquestador de I/O de alto rendimiento. Utiliza `BufWriter` para envolver la salida estándar, minimizando las llamadas al sistema operativo (`syscalls`) mediante el bloqueo del flujo de salida (`stdout.lock()`) y el uso de un búfer intermedio que mejora significativamente la eficiencia de la escritura en consola.

Nivel Decisorio (collatz): Funciona como un núcleo de cómputo puro. Gracias a la semántica de expresiones de Rust, el valor de retorno se omite sin palabra reservada (el `return` es implícito al omitir el punto y coma final), demostrando la eficiencia funcional del lenguaje al evitar bifurcaciones de retorno innecesarias.

Nivel de Medición (benchmark_collatz): Implementa un control preciso mediante la estructura `Instant` del módulo `std::time`, proporcionando una medición de intervalos de alta fidelidad. La gestión de memoria se externaliza mediante la librería `stats_alloc` y un `global_allocator`, permitiendo una inspección precisa del heap (`bytes_allocated`) sin incurrir en la sobrecarga interpretativa presente en otros lenguajes.

Análisis sintáctico de Zig

Zig se caracteriza por una arquitectura de diseño centrada en la transparencia semántica y la ausencia total de control de flujo implícito. A diferencia de los lenguajes de alto nivel que ocultan la gestión de recursos bajo abstracciones automáticas, la gramática de Zig impone una sintaxis explícita, donde cada operación de memoria o manejo de errores debe ser declarado de manera directa. Su analizador sintáctico procesa una estructura que favorece la legibilidad mediante la simplicidad.

La arquitectura del programa en Zig se organiza en tres estratos funcionales que garantizan un control absoluto sobre el hardware y la memoria:

Nivel de Control (similarCollatz): Actúa como el motor ejecutor de bajo nivel. A diferencia de otros lenguajes, este nivel delega el manejo del flujo de salida (writer) como un parámetro genérico (anytype), permitiendo una flexibilidad polimórfica que elimina la sobrecarga de despacho dinámico.

Nivel Decisorio (collatz): Constituye el núcleo matemático del algoritmo. La sintaxis enfatiza la pureza de la función, retornando el valor calculado de forma directa tras una evaluación de paridad simple. Su estructura previene conversiones implícitas, garantizando que el procesador ejecute las operaciones aritméticas sobre tipos enteros de tamaño fijo (u64).

Nivel de Medición (main): Implementa un control de máxima precisión mediante la inyección explícita de un asignador de memoria (GeneralPurposeAllocator). Este nivel destaca por su transparencia operativa: las mediciones de tiempo se obtienen directamente mediante marcas temporales de nanosegundos (nanoTimestamp), y el consumo de RAM se audita consultando las métricas de estado internas del asignador (total_requested_bytes). Este diseño asegura que no existan fugas de memoria ni residuos de recolección de basura que alteren los datos del benchmark.

Benchmarking y Tabla de Tiempos

Para garantizar la reproducibilidad y la validez de las mediciones de rendimiento presentadas en esta sección, se ha estandarizado un entorno de pruebas consistente en un computador equipado con un procesador Intel i5 7500, 8 GB de memoria RAM DDR4 y una unidad de almacenamiento SSD NVMe de 512 GB. El sistema operativo base utilizado es

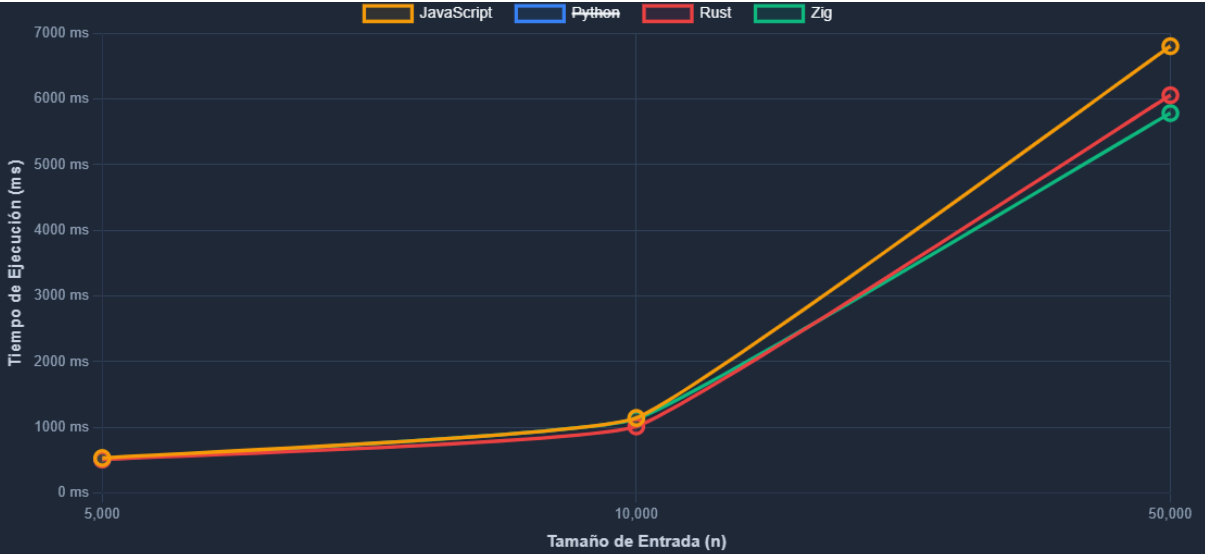
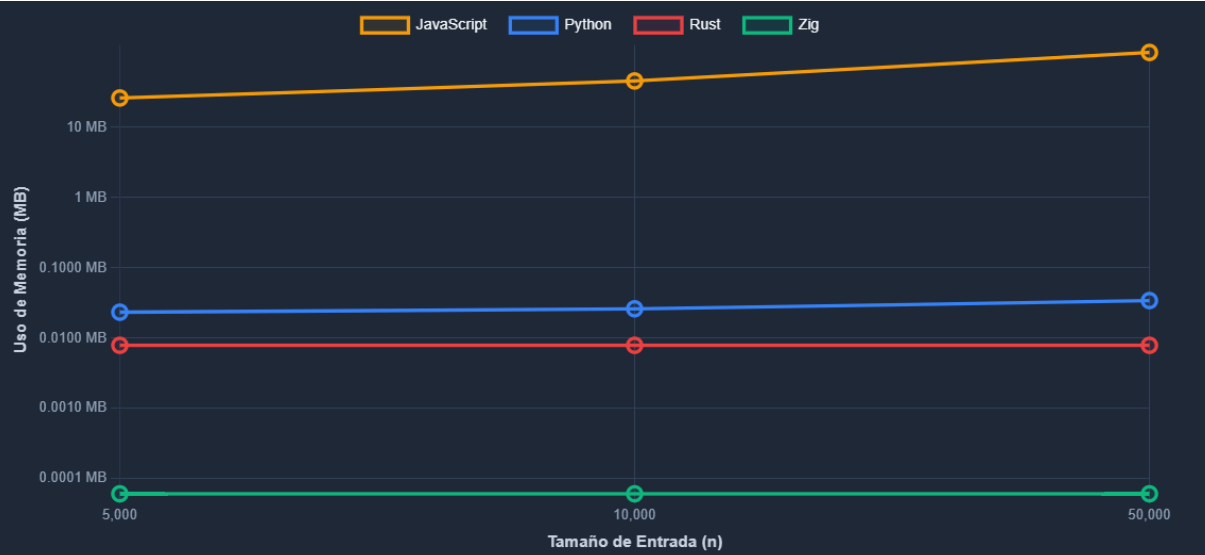
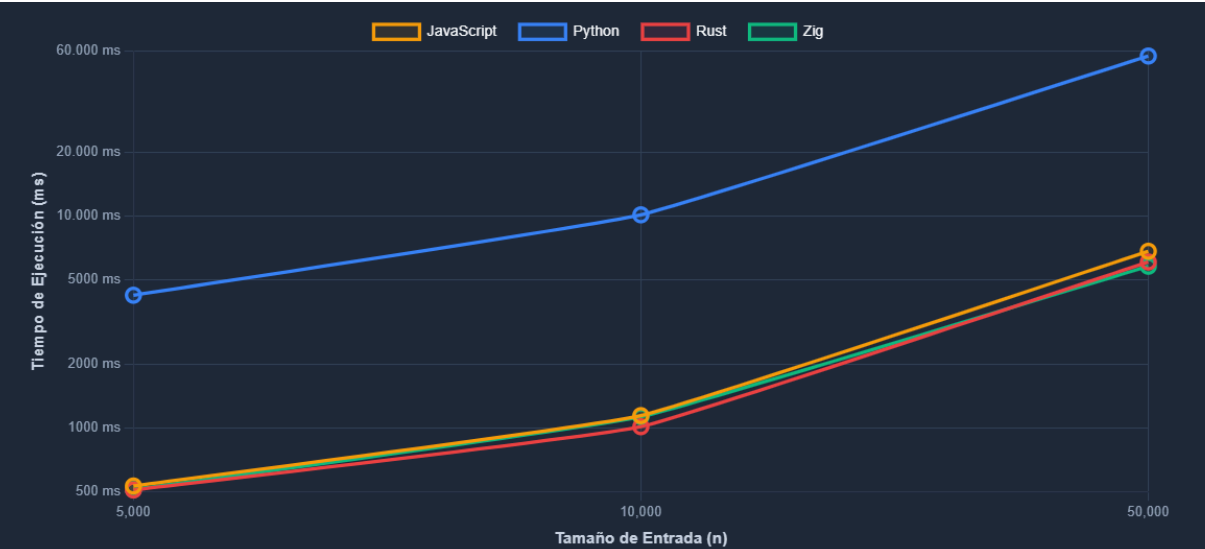
Windows 10, y el entorno de desarrollo y ejecución de los algoritmos ha sido configurado íntegramente sobre Visual Studio Code.

Para asegurar la consistencia operativa y la fidelidad de las métricas obtenidas, se han definido las siguientes versiones de los entornos de ejecución y sus respectivas directivas de compilación y despliegue:

- **Zig (v>0.13.0):**zig build run -Doptimize=ReleaseFast
- **Python (v>3.11.7):**py .\main.py
- **JavaScript (Node.js v>22.16.0):**node .\main.js
- **Rust (Cargo v>1.96.0):**cargo run --release

Tabla de resultados y graficos benchmarking

Lenguaje	Paradigma	Mecanismo de ejecución	Operaciones (n)	Tiempo Promedio (ms)	RAM Pico (MB)
JavaScript	Multiparadigma (Interpretado)	Node.js (V8 / JIT)	5000	532,0149	26,126051
			10000	1.142,2960	45,665901
			50000	6.804,8727	116,140936
Python	Multiparadigma (Interpretado)	CPython	5000	4.222,5967	0,023397
			10000	10.111,1300	0,026167
			50000	56.718,6152	0,034247
Rust	Multiparadigma (Compilado)	Cargo (LLVM)	5000	509,4000	0,007910
			10000	1.012,8000	0,007910
			50000	6.057,9000	0,007910
Zig	Imperativo (Compilado)	Zig Build (LLVM)	5000	511,6647	0,000061
			10000	1.127,6699	0,000061
			50000	5.785,3608	0,000061



Tras un total de 10 iteraciones del algoritmo en cada lenguaje se obtuvo un tiempo de ejecución y consumo de memoria pico promedio. Como se puede observar, Rust y Zig son los más veloces y eficientes, teniendo un consumo de memoria fijo en contraste con Python o JavaScript, y tiempos de respuesta menores a 10 segundos incluso tratándose de 50000 números. El ejercicio tomó en cuenta la capacidad que tiene el buffer de escritura de Python que por defecto es mucho más optimizado, y se implementó en el resto de lenguajes para igualar su velocidad de cálculo con la velocidad de escritura en consola por cada lenguaje.

JavaScript a pesar de competir con los lenguajes compilados en velocidad, su alto consumo de memoria es un tema a tomar en consideración en entornos limitados, debido a que debe simular una estructura de tipo web en cada operación que realiza incluso implementando Node.JS para no depender de algún navegador, esto debido a su propia naturaleza. El motor V8 de Node.js emplea mecanismos de optimización especulativa y compilación Just-In-Time (JIT) que, si bien reducen la latencia computacional a niveles cercanos a Rust, lo hacen a costa de una dispersión exponencial de la memoria alojada. Esta voracidad se debe a que V8 retiene múltiples versiones del mismo fragmento de código, incluyendo el árbol de sintaxis abstracta, tablas de símbolos y el bytecode optimizado, elevando el consumo por encima de los 100 MB en cargas intensivas.

En contraste, la hegemonía técnica de Rust y Zig radica en su arquitectura de compilación Ahead-Of-Time (AOT), que permite transformar la matriz lógica en código binario puro sin la sobrecarga de un recolector de basura o máquinas virtuales intermedias. Mientras que Python exhibe una latencia térmica significativa debido a la instanciación masiva de estructuras complejas como el PyLongObject para cada operación aritmética, Rust y Zig confinan el procesamiento exclusivamente a los registros y la pila estática, alcanzando lo que se define como abstracciones de costo cero.

Actividad III: Diseño de un Lenguaje de Dominio Específico (DSL)

para Sistemas lógico - físicos (HMI)

El desarrollo de sistemas lógicos-físicos críticos, como la microrred eléctrica inteligente y el sistema de almacenamiento de energía *ECO-GRID*, exige herramientas de software que mitiguen el error humano y garanticen la seguridad operativa en tiempo real. Con este propósito, se propone el diseño conceptual del Lenguaje L, un Lenguaje de Dominio Específico (DSL) de paradigma imperativo y estructurado, ideado específicamente para ser interpretado por operarios de planta sin necesidad de poseer destrezas avanzadas en programación de propósito general.

El enfoque primordial de este lenguaje radica en abstraer la complejidad técnica de los controladores de bajo nivel (sensores térmicos, inversores y actuadores mecánicos) mediante una sintaxis limpia, intuitiva y robusta. A continuación, se detallará la especificación formal de este entorno a través de su arquitectura léxica (alfabeto y clasificación de tokens), su catálogo de palabras reservadas operacionales y la estructura de su gramática sintáctica abstracta, validando finalmente su expresividad computacional mediante escenarios prácticos de contingencia industrial.

Alfabeto y reglas léxicas

La fase inicial en el diseño de la arquitectura del compilador para el Lenguaje L requiere la delimitación matemática de su frontera morfológica. Esta sección define el *Alfabeto Aceptado*, el cual restringe los caracteres válidos que el analizador léxico procesa para evitar la introducción de símbolos espurios en el sistema. Asimismo, se establece la política de *Componentes Léxicos (Tokens)*, especificando las expresiones regulares y los patrones conceptuales que agrupan estas cadenas en unidades con significado lógico y semántico para la planta.

- **Caracteres Alfabéticos:** Letras mayúsculas y minúsculas del alfabeto estándar [a-z, A-Z], destinadas a la construcción de palabras reservadas e identificadores.
- **Caracteres Numéricos:** Dígitos enteros del [0-9] empleados en la declaración de identificadores de hardware, constantes de carga y umbrales de temperatura.

- **Símbolos Operacionales y de Puntuación:** * Operador de asignación, comillas simples (‘) y dobles (“) para las alerta o mensajes planos, y comparación relacional: Igualdad (=), menor que (<), y mayor que (>).
 - ➔ *Delimitadores de parámetros de funciones:* Paréntesis de apertura (y cierre).
 - ➔ *Separador de argumentos:* Carácter coma (,).
- **Caracteres de Espaciado:** Espacios en blanco () y saltos de línea (\n), utilizados como separadores morfológicos y estructuradores de sintaxis.

Componentes Léxicas (Tokens)

El analizador léxico agrupará las cadenas del alfabeto en unidades con significado propio denominadas tokens, clasificadas de la siguiente manera:

Tabla Componentes léxicos

Componente Léxico (Token)	Descripción Técnica y Semántica	Expresión Regular Conceptual (Patrón)	Ejemplos en el Dominio ECO-GRID
ID	Nombres de variables y dispositivos asignados por el operario. Debe iniciar en minúscula y pueden contener guiones bajos para separar términos.	[a-z][a-z0-9_]*	bateria_litio_1, sensor_celda_a
NUM	Constantes numéricas de precisión para temperaturas, potencias o porcentajes de carga.	[0-9]+(\.[0-9]+)?	55, 90, 55.5, 20
ESTADO	Constantes literales industriales para el control directo de actuadores y relés.	ON OFF	ON, OFF
STRING	Constantes de texto para el despliegue de mensajes informativos y alarmas.	"[^\"]\\\\"*	"fuga térmica crítica"
KEYWORD	Primitivas del sistema y palabras sagradas de control de flujo arquitectónico.	Cadenas fijas explícitas	init_grid, si, mientras, conmutar_linea
OP_REL	Operadores lógicos para asignaciones de estados y la evaluación lógica de condiciones matemáticas en las estructuras de control.	= < >	=, <, >
SYM	Delimitadores de sintaxis para encapsulamiento y separación de argumentos de funciones.	() ,	(,), ,
BOOL	Constantes lógicas de control para bucles de monitoreo continuo.	verdadero falso	verdadero, falso

Con el fin de proveer una sintaxis limpia y moderna orientada a la eficiencia del operario, se descarta el uso del punto y coma. **El Salto de Línea (\n)** es establecido como el delimitador formal único que le indica al analizador la culminación de una instrucción válida.

Palabras Clave Obligatorias

Estas cadenas de caracteres alfabéticos están predefinidas en el analizador sintáctico y poseen un significado unívoco que no puede ser modificado ni utilizado por el operario para la declaración de identificadores comunes (ID). Estas palabras clave se clasifican en:

Primitivas de Inicialización e Interfaz de Hardware

Son las instrucciones nativas que interactúan directamente con los dispositivos físicos de la planta. Estas son:

- **init_grid:** Palabra reservada de ejecución imperativa obligatoria que inicializa la arquitectura lógica del sistema, calibra los sensores y establece la conexión segura con los relés de alta potencia. Debe invocarse al inicio de todo script operativo.
- **leer_temperatura:** Primitiva de lectura que recibe como argumento un identificador de hardware (*bateria_id*) entre paréntesis. Se comunica de forma directa con los sensores térmicos de celda y retorna un literal numérico (*NUM*) con el valor en grados Celsius del contenedor analizado.
- **estado_carga:** Primitiva de monitoreo que recibe como parámetro el identificador de un banco de almacenamiento (*bateria_id*). Consulta de forma síncrona el estado actual de los inversores y los sensores de flujo eléctrico, retornando un valor numérico entero del 0 al 100 que representa el porcentaje de carga remanente.
- **conmutar_linea:** Primitiva de acción y control directo sobre los actuadores mecánicos de la planta. Recibe dos argumentos mandatorios separados por una coma: el identificador del sector industrial a gestionar (*sector_id*) y el literal de estado operativo deseado, el cual acepta únicamente los tokens *ON* o *OFF* para conectar o aislar el flujo de potencia.

- **emitir_alerta:** Función nativa de diagnóstico y seguridad industrial diseñada para despachar mensajes a la interfaz hombre-máquina (*HMI*). Recibe como argumento único un literal de cadena de texto (*STRING*) encerrado entre comillas dobles y se utiliza para disparar alarmas recurrentes en la consola de control ante contingencias operativas.

Estructuras de Control de Flujo Lingüístico

Para garantizar la seguridad operacional en la microrred, el Lenguaje L implementa estructuras de control de flujo de naturaleza imperativa. A diferencia de un enfoque puramente de hardware, este DSL separa de forma explícita los valores de verdad lógicos (Evaluación Semántica mediante el token *BOOL*) de los estados físicos de los actuadores (tokens *ESTADO* como *ON/OFF*). Con esto, las bifurcaciones y ciclos evalúan proposiciones lógicas puras, garantizando un flujo estructurado y seguro.

- **Estructura Condicional (*si / entonces / fin_si*):** permite la separación del flujo de ejecución basándose en una proposición lógica determinista. Su sintaxis abstracta exige que la palabra reservada *si* vaya seguida de una *<Expresion>* (la cual puede evaluar directamente un token *BOOL* o el resultado de una relación matemática entre *<Termino>*s). Si la expresión se evalúa como verdadero, el motor de ejecución procesa la *<ListaSentencias>* interna de forma secuencial hasta alcanzar el delimitador de cierre formal *fin_si*.
- **Estructura Iterativa (*mientras / ejecutar / fin_mientras*):** Bloque de control cíclico diseñado para el monitoreo continuo, en tiempo real e indefinido de las variables analógicas de **ECO-GRID**. Diseñada específicamente para los bucles de monitoreo continuo y en tiempo real que exige la planta industrial. Su orden sintáctico mandatorio exige la palabra reservada *mientras*, una condición lógica del tipo *<Expresion>*, y la palabra clave *ejecutar*.

Gramática Sintáctica Abstracta

Para garantizar que el analizador sintáctico del **Lenguaje L** interprete los comandos de control de la microrred de manera jerárquica y sin ambigüedades, se establece la

especificación formal de las reglas de producción que gobiernan el lenguaje. Las reglas de producción estructurales son:

- **Estructura General del Programa (<Programa>):** Todo script operativo válido en Lenguaje L debe iniciar obligatoriamente con la primitiva de inicialización *init_grid*, seguida inmediatamente por un delimitador de salto de línea (`\n`) y un cuerpo compuesto por una secuencia consecutiva de una o más sentencias válidas.
- **Definición de Sentencia (<Sentencia>):** Representa una línea de instrucción individual. Toda sentencia debe estar aislada por un salto de línea y se clasifica sintácticamente en: una asignación de datos, una llamada a una función nativa de hardware, una estructura de bifurcación condicional o un bucle de monitoreo repetitivo.
- **Sintaxis de la Estructura Condicional (<SentenciaCondicional>):** Establece un bloque de ejecución condicional. Debe respetar de forma estricta el siguiente orden de tokens: la palabra reservada *si*, una condición lógica evaluable (<Expresion>), la palabra reservada *entonces*, un salto de línea, un bloque interno de sentencias operativas, y la palabra reservada de clausura *fin_si*.
- **Sintaxis de la Estructura Iterativa (<SentenciaIterativa>):** Regula los bucles de vigilancia continua de **ECO-GRID**. Su orden sintáctico mandatorio es: la palabra reservada *mientras*, una condición o constante lógica (<Expresion>), la palabra reservada *ejecutar*, un salto de línea, un bloque de sentencias cíclicas y la palabra de cierre formal *fin_mientras*.
- **Regla de Expresiones Lógicas (<Expresion>):** Define cómo se realizan las evaluaciones en los condicionales o bucles. Se compone de un identificador de variable (ID) o una función de lectura analógica, un operador relacional (*OP_REL* como *<*, *>* o *=*) y una constante de comparación que puede ser un literal numérico (*NUM*) o un estado operativo fijo (*ESTADO*).
- **<Termino> en Expresiones Lógicas:** la gramática abstracta evoluciona para agrupar bajo la regla <Termino> a cualquier elemento capaz de producir u otorgar un valor computable por el analizador sintáctico: identificadores (ID), constantes numéricas (NUM), estados de relés (ESTADO), constantes lógicas (BOOL) o funciones analógicas de lectura (<LlamadaFuncion>).

Formalización en Pseudo-Notación Gramatical

Para que el compilador conceptual pueda revisar el código, utiliza esta “receta formal” (llamada notación BNF). Ejemplo:

```
<Programa> ::= "init_grid" "\n" <ListaSentencias>
<ListaSentencias> ::= <Sentencia> | <Sentencia> <ListaSentencias>
<Sentencia> ::= <Asignacion> "\n"
                | <LlamadaFuncion> "\n"
                | <SentenciaCondicional> "\n"
                | <Sentencialterativa> "\n"
<SentenciaCondicional> ::= "si" <Expresion> "entonces" "\n" <ListaSentencias>
"fin_si"
<Sentencialterativa> ::= "mientras" <Expresion> "ejecutar" "\n" <ListaSentencias>
"fin_mientras"
<Asignacion> ::= ID "=" <Expresion>
                | ID "=" <LlamadaFuncion>
<LlamadaFuncion> ::= "leer_temperatura" "(" ID ")"
                | "estado_carga" "(" ID ")"
                | "conmutar_linea" "(" ID "," ESTADO ")"
                | "emitir_alerta" "(" STRING ")"
<Expresion> ::= <Termino> OP_REL <Termino>
                | BOOL
<Termino> ::= ID
                | NUM
                | ESTADO
                | BOOL
                | <LlamadaFuncion>
```

Escenarios Operativos

Para demostrar la viabilidad y expresividad del DSL, a continuación se presentarán dos programas que darán respuesta a los siguientes escenarios operativos específicos de la planta ECO-GRID:

- **Escenario Operativo A (Prevención de Fuga Térmica y Gestión de Alivio de Carga):** El programa debe monitorizar de forma continua e iterativa la temperatura de un banco de baterías específico empleando estructuras repetitivas. Si la temperatura registrada supera un umbral crítico de seguridad (por ejemplo, 55 grados Celsius), el programa debe activar de forma inmediata los sistemas de refrigeración auxiliar, desconectar las líneas de carga solar para detener el ingreso de energía térmica y desviar el consumo del sector industrial hacia la red comercial de respaldo para aliviar el estrés del componente. El ciclo debe terminar o emitir alertas de emergencia recurrentes si el peligro persiste tras un período determinado.

init_grid

mientras verdadero ejecutar

temp_bateria = leer_temperatura(bateria_litio_1)

si temp_bateria > 55 entonces

conmutar_linea(sistema_refrigeracion, ON)

conmutar_linea(paneles_solares_entrada, OFF)

conmutar_linea(sector_industrial, OFF)

conmutar_linea(red_comercial_respaldo, ON)

emitir_alerta("ALERTA: Fuga térmica crítica detectada. Carga desviada.")

fin_si

fin_mientras

Escenario Operativo B (Balance de Carga y Optimización Energética): el siguiente script evaluará el estado de carga actual de los sistemas de almacenamiento de la planta. Si el nivel de carga de la batería es óptimo (superior al 90%) y la generación de los paneles solares excede la demanda interna actual detectada por los sensores de flujo, se ejecutan comandos para accionar los relés automáticos y vender/inyectar el excedente energético hacia la red eléctrica pública general. En caso contrario, si las baterías descienden de un límite crítico (menor al 20%) durante horas de alta demanda nocturna, se aíslan los sectores no esenciales de la fábrica para preservar el suministro en las áreas médicas o de servidores críticos de la organización.

init_grid

```
carga_actual = estado_carga(bateria_respaldo_c2)
si carga_actual > 90 entonces
    conmutar_linea(rele_inyeccion_publica, ON)
    emitir_alerta("INFO: Baterías óptimas. Inyectando excedente a la red pública.")
fin_si
si carga_actual < 20 entonces
    conmutar_linea(sector_fabrica_no_esencial, OFF)
    conmutar_linea(sector_servidores_criticos, ON)
    conmutar_linea(sector_medico, ON)
    emitir_alerta("ALERTA: Carga crítica en horas pico. Sectores no esenciales
aislados.")
fin_si
```

Conclusión

Mediante el presente informe a través de las tres fases desarrolladas, ha sido posible establecer una correlación directa entre las decisiones de diseño léxico-sintáctico y la eficiencia operativa final del software.

En primer lugar, el examen de los paradigmas de programación permitió comprender que las corrientes imperativas, orientadas a objetos, funcionales, lógicas y concurrentes no deben ser interpretadas como límites estáticos, sino como modelos arquitectónicos que condicionan la gestión del estado y la memoria, demostrando que el paradigma personal del ingeniero es tan determinante como el lenguaje elegido.

La comparativa entre lenguajes interpretados como Python o JavaScript y compilados como Rust o Zig reveló que la abstracción tiene un costo; mientras que los entornos con recolectores de basura y máquinas virtuales facilitan la productividad mediante una sintaxis laxa y dinámica, los lenguajes compilados bajo el modelo Ahead-Of-Time demuestran que, al prescindir de intermediarios y aplicar una gestión determinista de los recursos, es posible alcanzar el mínimo costo, tratándose acerca de que la eficiencia del procesador y la estabilidad de la memoria no son accidentes, sino consecuencias directas de reglas morfológicas y sintácticas que obligan a la claridad del flujo y al tipado estático, minimizando las ambigüedades durante la traducción al código máquina.

Finalmente, el diseño del Lenguaje L para la automatización de la microrred inteligente ECO-GRID materializó la importancia de la teoría de compiladores en la seguridad industrial. La capacidad de diagnosticar por qué un lenguaje es más lento, más seguro o más expresivo que otro es lo que permite, en última instancia, elegir la herramienta correcta para el desafío técnico correcto. La programación, entendida como ciencia, es la disciplina de gestionar la abstracción sin perder el control sobre la realidad física de la máquina.

Referencias bibliográficas

- Tomasetti, F. (2017, 20 de febrero). The complete guide to (external) Domain Specific Languages. Recuperado de: <https://tomasetti.me/domain-specific-languages/>
- Python España. (2016, 2 de febrero). Introducción a los DSL (Domain Specific Languages) en Python - J. Ignacio Rodríguez - Track Básico. Recuperado de: <https://www.youtube.com/watch?v=nZl2FluQxiM>
- Espínola, J. P. S. (2025, 12 septiembre). Paradigma - Qué es, tipos y características. Recuperado de <https://concepto.de/que-es-paradigma/>
- Miriam. (2026, 24 febrero). ¿Qué son los paradigmas de programación? Recuperado de <https://profile.es/blog/que-son-los-paradigmas-de-programacion/>
- HostingPlus Mexico. (2021, 4 noviembre). Paradigmas de programación: qué son y sus diferentes tipos | Blog | Hosting Plus Estados Unidos. Recuperado de <https://www.hostingplus.lat/blog/paradigmas-de-programacion-que-son-y-sus-diferentes-tipos/>